FocusFlow AI

Multi-Agent Personal Productivity Assistant

Project Specification and Implementation Plan

Prepared for submission

Project Summary

FocusFlow AI is a personal assistant that asks for the user's daily to-do list, reads calendar events, finds free time, schedules tasks around fixed meetings, and tracks weekly, monthly, and yearly goals. The system uses FastAPI, LangGraph, LangChain, SQLite, and free local LLMs through Ollama for the MVP, with Google Calendar integration added after the mock calendar version is complete.

Table of Contents

1. Project Overview

FocusFlow AI is an AI-powered multi-agent personal productivity assistant. It helps users plan their day by combining natural language task input, calendar availability, task priority, deadlines, and long-term goals.

Project Name	FocusFlow AI
Project Type	AI-powered productivity and scheduling assistant
Primary Backend	FastAPI with REST APIs
Agent Framework	LangGraph with LangChain
LLM Strategy	Free local models through Ollama for MVP; provider-agnostic design for OpenAI/Claude-compatible APIs later
Database	SQLite for MVP; PostgreSQL planned for production
Primary Integration	Mock calendar first, Google Calendar API in Version 2

2. Problem Statement

Students, job seekers, researchers, and working professionals often struggle to plan their day because tasks, meetings, deadlines, and long-term goals are stored in separate places. Manual planning takes time and often ignores real calendar availability. FocusFlow AI solves this by creating a daily plan around fixed calendar blocks and flexible work tasks.

3. Goals and Non-Goals

Goals

- Ask the user for a daily to-do list every morning.
- Extract structured tasks from natural language input.
- Pull or mock calendar events and treat them as fixed busy blocks.
- Find free time between calendar events.
- Schedule tasks into available slots based on priority, deadline, duration, and goal relevance.
- Validate the plan to avoid overlaps, conflicts, and unrealistic schedules.
- Track progress for daily tasks and weekly, monthly, and yearly goals.
- Provide an agent log that explains how the daily plan was generated.

Non-Goals for MVP

- No Gmail, Outlook, Google Maps, or mobile push notifications in the first MVP.
- No paid OpenAI or Claude dependency for the first MVP.
- No automatic calendar writes without user confirmation.
- No complex multi-user OAuth system in the first MVP.

- No voice assistant or mobile app in the first MVP.

4. Fixed MVP Scope

Feature	MVP Status	Notes
Daily morning check-in	Included	Dashboard notification or scheduled backend job
Natural language to-do extraction	Included	Uses local LLM through Ollama
Task CRUD	Included	Create, list, update, delete tasks
Goal CRUD	Included	Weekly, monthly, yearly goals
Mock calendar events	Included	Used before Google Calendar integration
Free-time detection	Included	Rule-based Python logic
Daily plan generation	Included	Schedules tasks into free slots
Validation and self-correction	Included	Detects conflicts and retries
Agent logs	Included	Shows each agent step
Google Calendar API	Version 2	Read events first, write focus blocks later
Gmail, Contacts, Maps	Future	Not part of MVP

5. Technology Stack

Layer	Technology	Reason
Backend API	FastAPI	Clean REST APIs, Swagger UI, Pydantic validation, Python ecosystem
Agent Orchestration	LangGraph	Graph-based multi-agent workflows with controlled state transitions
LLM Framework	LangChain	Common interface for local and cloud LLM providers
Free LLM	Ollama + Llama 3.1 8B or Qwen2.5 7B	Runs locally without paid APIs
Database	SQLite	Simple MVP database with easy local setup
ORM	SQLModel or SQLAlchemy	Structured relational models and queries
Validation	Pydantic	Typed request and response schemas
Scheduled Jobs	APScheduler	Daily morning check-in notification job
Frontend	React + Tailwind CSS	Simple dashboard UI
Charts	Recharts	Goal progress and productivity charts
Calendar Integration	Google Calendar API	Read fixed events and optionally create focus blocks later
Deployment	Docker later	Portable local and cloud deployment

Important design decision: AI is used for natural language understanding, summarization, explanation, and replanning suggestions. Deterministic Python logic is used for calendar conflict checking, free-time detection, task sorting, validation, and database updates.

6. System Architecture

```
User
 |
React Frontend / Swagger UI
 |
FastAPI REST API
 |
LangGraph Supervisor Workflow
 |
 ------------------------------------------------------------------
 |            |             |              |               |        |
Daily Agent  Task Agent    Calendar Agent  Free-Time Agent Planning Agent
 |
Validation Agent
 |
Goal Agent + Agent Logs
 |
SQLite Database
```

7. Multi-Agent Design

Agent	Responsibility	Output
Daily Agent	Receives the morning to-do message and prepares it for task extraction.	Raw daily intent and extracted task candidates
Task Agent	Creates, updates, and prioritizes tasks.	Structured task records
Calendar Agent	Reads fixed events from mock calendar or Google Calendar.	Busy blocks for the day
Free-Time Agent	Finds open slots between busy calendar blocks.	Available free slots
Planning Agent	Assigns tasks to free slots based on deadline, priority, duration, and goals.	Draft daily plan
Validation Agent	Checks conflicts, overlaps, missing breaks, and working-hour violations.	Valid/invalid result with reason
Goal Agent	Updates progress for weekly, monthly, and yearly goals.	Goal progress updates

8. Core User Workflows

Workflow 1: Morning Daily Planning

- System shows: Good morning! What are your top tasks for today?
- User enters tasks in natural language.
- Daily Agent extracts structured tasks.
- Calendar Agent pulls fixed calendar events.
- Free-Time Agent identifies available work slots.
- Planning Agent creates a daily schedule.

- Validation Agent checks the schedule.
- Goal Agent updates progress if tasks are completed.

Workflow 2: Goal Tracking

- User creates a weekly, monthly, or yearly goal.
- Tasks can be linked to a goal.
- When a task is completed, the related goal progress updates.
- Dashboard shows current value, target value, and progress percentage.

Workflow 3: Calendar-Aware Planning

- Calendar events are treated as fixed busy blocks.
- Flexible tasks are placed only in free slots.
- The system avoids overlapping tasks with meetings, classes, gym, or appointments.
- In Version 2, Google Calendar events replace mock calendar events.

9. Database Design

The MVP uses SQLite with the following tables. PostgreSQL can replace SQLite later without changing the core application logic.

Table	Fields
users	id, name, email, created_at
tasks	id, user_id, title, description, priority, deadline, estimated_minutes, status, goal_id, created_at, completed_at
goals	id, user_id, title, goal_type, target_value, current_value, unit, start_date, end_date, status
calendar_events	id, user_id, title, start_time, end_time, source
daily_plans	id, user_id, plan_date, status, created_at
daily_plan_items	id, plan_id, task_id, start_time, end_time, reason, status
agent_logs	id, agent_name, action, input_summary, output_summary, status, created_at
notifications	id, user_id, message, type, status, created_at, read_at

10. REST API Design

```
GET  /

POST /tasks
GET  /tasks
GET  /tasks/{id}
PATCH /tasks/{id}
DELETE /tasks/{id}

POST /goals
GET  /goals
GET  /goals/{id}
PATCH /goals/{id}
DELETE /goals/{id}
GET  /goals/progress
```

```
GET  /calendar/events?date=YYYY-MM-DD
GET  /calendar/free-slots?date=YYYY-MM-DD

POST /daily/checkin
POST /daily/plan
GET  /daily/plan?date=YYYY-MM-DD

POST /assistant/daily-plan

GET  /dashboard/today
GET  /agent-logs
GET  /notifications
PATCH /notifications/{id}
```

11. Scheduling and Validation Logic

Scheduling Algorithm

- Get fixed calendar events for the selected date.
- Calculate free slots between working hours and fixed events.
- Sort tasks by deadline, priority, estimated duration, and goal importance.
- Place tasks into free slots where they fit.
- Split large tasks into smaller blocks if needed.
- Add breaks between focus blocks.
- Save the final plan and return it to the user.

Validation Rules

- No task overlaps with a fixed calendar event.
- No task overlaps with another task.
- No task is scheduled outside working hours.
- Tasks must fit inside available slots.
- High-priority and urgent tasks should be scheduled earlier.
- Breaks should be inserted between long work blocks.
- If a plan fails validation, the planner retries with another slot up to two times.

12. Implementation Iterations

Iteration	Focus	Deliverables	Done When
0	Project Setup	FastAPI app, SQLite connection, folder structure, .env, Swagger UI	API runs at /docs
1	Task and Goal System	Task CRUD, Goal CRUD, goal progress updates	Task completion updates related goal progress
2	Mock Calendar + Free-Time Finder	Mock events, calendar endpoints, free slot logic	Backend returns fixed events and free slots
3	Daily Plan Generator	Planner service, daily	Tasks are scheduled into

		plans, plan items	free slots
4	Validation + Self-Correction	Validation agent, retry logic, agent logs	Invalid plans are corrected automatically
5	AI Daily Check-in	Ollama LLM task extraction endpoint	Natural language input creates structured tasks
6	LangGraph Multi-Agent Workflow	Supervisor graph connecting all agents	Single endpoint generates a full daily plan
7	Morning Check-in Reminder	APScheduler job, notification table	Dashboard shows morning check-in prompt
8	Frontend Dashboard	React UI, daily plan, tasks, goals, progress, agent logs	Demo can be recorded end-to-end
9	Google Calendar Read Integration	OAuth/test credentials, read real calendar events	Real events block daily planning
10	Optional Focus Block Creation	Confirm-before-write calendar focus blocks	User can approve focus blocks before creation

13. Testing Strategy

- **Unit tests:** free-time calculation, overlap detection, task sorting, goal progress calculation.
- **API tests:** task endpoints, goal endpoints, daily check-in, daily plan generation, dashboard response.
- **Agent tests:** verify each agent receives expected input and returns expected state updates.
- **Validation tests:** calendar conflict, task overlap, missing free time, oversized task, outside working hours.
- **Demo tests:** run a full daily check-in and verify that a usable daily plan is produced.

14. Future Enhancements

- Gmail integration for meeting preparation reminders and email summaries.
- Google Contacts integration for resolving people mentioned in tasks or meetings.
- Google Maps integration for location-aware errands and travel-time-aware planning.
- Outlook Calendar integration.
- Mobile push notifications.
- Voice input for daily check-ins.
- PostgreSQL and multi-user authentication.
- Advanced analytics for productivity trends.

15. Resume Positioning

After implementation, the project can be described using the following resume bullets:

- Developed FocusFlow AI, a multi-agent personal productivity assistant using FastAPI, LangGraph, LangChain, and local LLMs to generate daily schedules from natural language to-do lists, calendar events, and user goals.

- Built specialized agents for daily check-ins, task extraction, calendar blocking, free-time detection, planning, validation, self-correction, and weekly/monthly/yearly goal-progress tracking.
- Integrated rule-based scheduling with LLM-assisted task understanding to create conflict-free daily plans around fixed calendar events, reducing manual planning effort and improving schedule feasibility.

16. References

- FastAPI SQL database documentation: https://fastapi.tiangolo.com/tutorial/sql-databases/
- LangGraph workflows and agents documentation: https://docs.langchain.com/oss/python/langgraph/workflows-agents
- LangGraph multi-agent workflows overview: https://www.langchain.com/blog/langgraph-multi-agent-workflows
- LangChain Ollama integration documentation: https://docs.langchain.com/oss/python/integrations/providers/ollama
- Google Calendar API Python quickstart: https://developers.google.com/workspace/calendar/api/quickstart/python
- Google Calendar API overview: https://developers.google.com/workspace/calendar/api/guides/overview